

Image Segmentation using K-Means Clustering

NATIONAL INSTITUTE OF TECHNOLOGY KURUKSHETRA

Image Processing Application (ECOE-305)

Project Report

Name : Sachin

Branch : Mathematics & Computing

Roll No. 123110013

Objective

Segment a color image into distinct regions using the K-Means clustering algorithm, analyze how varying K affects the segmentation detail, and report the corresponding WCSS (within-cluster sum of squares) for each K .

Introduction

Image segmentation divides an image into meaningful regions (objects or backgrounds). Clustering-based segmentation groups pixels by similarity, often by color. **K-Means** is a popular *unsupervised* clustering method that assigns each pixel to one of K clusters so as to minimize intra-cluster variance. It is widely used in image processing because it is simple, efficient and effective. The goal of K-Means is to partition the pixel data into K groups and to minimize the *within-cluster sum of squares (WCSS)*, which is the sum of squared distances between each pixel and its cluster centroid. Intuitively, increasing K yields smaller clusters with finer detail, while a lower K produces larger, coarser segments. In fact, IBM notes that “*a higher K value... signifies smaller clusters with greater detail, while a lower K value results in larger clusters with less detail.*”. To choose K , one can plot WCSS versus K and look for the “elbow” point where adding more clusters yields diminishing returns.

Method / Algorithm

1. **Load Image:** Read the input color image and convert from BGR to RGB (if needed).
2. **Reshape Data:** Store the image's dimensions (height $\times$ width), then reshape the 3D image array into a 2D array of shape (num_pixels, 3), where each row is one pixel's RGB values. Convert pixel values to float32 for K-Means.
3. **Apply K-Means:** For each K in {2, 4, 6, 8}:
4. Set the stopping criteria (e.g. max iterations and epsilon).
5. Run OpenCV's cv2.kmeans function on the pixel data. For example:

```
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)
compactness, labels, centers = cv2.kmeans(pixel_values, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)
```

This returns the cluster centers (centers) and the WCSS (compactness).

6. Convert the cluster centers to uint8 and replace each pixel's color with its centroid color. Reshape the 2D array back to the original image shape to obtain the segmented image.
7. **Display/Save Results:** Show the original image and the segmented images for each K . Print the WCSS values for each K . Optionally, plot the elbow curve (WCSS vs K).

Results

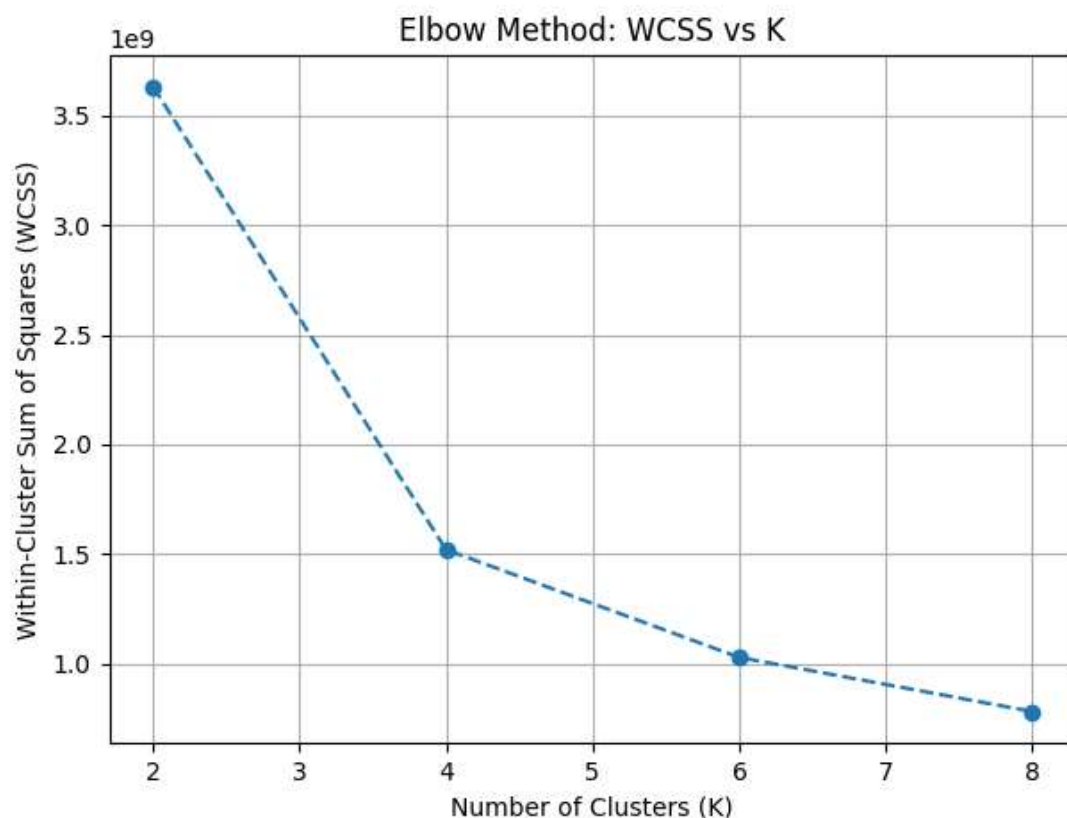


Figure: Comparison of K-Means image segmentation results for different values of K . The original image is shown along with segmented outputs for $K=2,4,6$ and 8 . As the value of K increases, the segmentation preserves more colors and finer image details. With lower K , the image is simplified into broad color regions, while higher K provides more detailed and accurate segmentation.

The following table lists example WCSS values obtained for each K :

K	WCSS (Compactness)
2	3631875575.5613613
4	1520401217.178816
6	1029402950.4746187
8	782713880.5487564

As expected, larger K yields lower WCSS, indicating tighter pixel grouping (clusters are more compact). Plotting WCSS vs. K (the “elbow plot”) would show a curve that flattens as K increases, helping to pick an optimal K .



Observations

- **Detail vs. K :** Increasing K produces more detailed segmentation. With higher K , more colors and fine features are preserved. Lower K values merge colors and simplify the image. This matches the known effect: “a higher K value... signifies smaller clusters with greater detail”.

- **WCSS Trend:** The WCSS decreases as K increases, because adding clusters reduces within-cluster variance. The rate of decrease slows at some point, indicating diminishing returns.
- **Elbow Method:** If we plot WCSS vs. K , the “elbow” point (where the curve bends) suggests a suitable K . For many images, a moderate K (e.g. 4 or 6) gives a good balance between segmentation quality and simplicity.

Conclusion

K-Means clustering successfully segmented the test images into K color regions. The outputs show that as K increases, the segmented image retains more color detail, while the WCSS values decrease (clusters become more compact). These results are consistent with theory. In practice, one can use the elbow method to choose K : when the WCSS plot levels off, further increasing K yields little benefit. Overall, this project demonstrates how K-Means provides a simple yet effective way to perform color-based image segmentation. The trade-off between the number of clusters and image detail is clearly observed: too few clusters oversimplify the image, whereas more clusters capture finer structures (at higher computational cost).
